

ExpenseOps

AI-Powered Expense Management Model Context Protocol (MCP) Server
Comprehensive Enterprise System Design Document (SDD)

Document Version	1.0.0 (Production Blueprint)
Date	May 21, 2026
Author	Principal Enterprise Infrastructure Architect & Platform Engineering Team
Target Audience	Solution Architects, Backend Engineers, Platform Teams, AI Integration Engineers
Status	APPROVED / READY FOR IMPLEMENTATION

Confidentiality & Usage Notice

This document contains proprietary architectural patterns and technical designs for ExpenseOps. Distribution or implementation without explicit authorization from the Platform Governance Board is prohibited. All implementations must adhere strictly to the NFRs and validation logic detailed herein.

1. Introduction

| 1.1 Purpose of the SDD

This System Design Document (SDD) establishes a formal, production-grade architectural blueprint for **ExpenseOps**, an AI-powered expense management server natively built on top of Anthropic's Model Context Protocol (MCP). This document defines the structural patterns, API schemas, data topologies, rule engine workflows, and security frameworks necessary to transition ExpenseOps from concept into a scalable, enterprise-ready fintech automation platform.

| 1.2 Project Overview

ExpenseOps transforms legacy, human-in-the-loop expense compliance processes into an autonomous workflow orchestrator. Natively utilizing MCP, the platform abstracts complex tasks—such as optical character recognition (OCR), multi-dimensional line-item parsing, policy enforcement, structural risk/fraud scoring, and real-time dashboard updating—into single, reusable tools. These tools are dynamically discoverable and invocable by Large Language Model (LLM) agents, internal microservices, and enterprise automation runners.

| 1.3 Intended Audience

- **Solution Architects:** To evaluate system compliance with enterprise security, isolation barriers, and governance standards.
- **Backend & Platform Engineers:** To serve as a definitive implementation guide for codebase construction, schema generation, and routing logic.
- **Enterprise Software Teams:** To facilitate seamless tool discovery, tool integration, and agent-driven corporate workflow authoring.

| 1.4 Technical Scope

The scope of this document covers the implementation details of the core Python-based server layer using `FastMCP`, the deterministic business rules engine, statistical risk/fraud calculations, local-to-cloud transactional database configurations, UI layout semantics, and fallback procedures for flaky AI model dependencies. Legacy integrations (such as SAP/Workday pipelines) are addressed via abstract data transfer definitions.

| 1.5 Engineering Objectives

The platform is architected to achieve extreme reliability in data capture, deterministic transparency in policy execution, low-latency API response times, clear audit trails, and strict separation between non-deterministic AI tasks and immutable compliance rules.

2. System Overview

2.1 Overall Platform Behavior

ExpenseOps functions as a dual-mode engine: it serves traditional requests via standard REST APIs while simultaneously operating as an active Model Context Protocol (MCP) server. When a user submits an expense via the frontend or an automated enterprise receipt sink, the data flows into a multi-tiered ingestion pipe. The system unifies raw text capture, multi-stage layout extraction, and predictive LLM token structuring into a single context-aware pass.

2.2 Finance Automation Objectives

Traditional finance departments suffer from human error, delayed processing windows, and high operational costs. ExpenseOps replaces physical manual matching with autonomous validations. By executing continuous vector-distance calculations for duplicates and parsing multi-line receipts against complex cross-border tax codes, the system processes 95% of standard receipts without human intervention, escalating only clear structural anomalies to human reviewers.

2.3 MCP-Native Architecture

Unlike standard monolithic wrappers, ExpenseOps treats the MCP specification as its primary protocol surface. The server registers its capabilities (tools, prompts, and resources) with an upstream MCP Host (such as a corporate proxy or an LLM client). This permits structural agent-driven exploration: an AI agent reading an email body can independently call ExpenseOps' tools to extract a receipt, validate it against corporate travel policy, flag exceptions, and present a structured summary directly to an executive.

2.4 Reusable Tooling Concept

Every internal subsystem is exposed as an atomic, stateless MCP tool. These tools are designed with zero cross-tool tight coupling; they rely entirely on uniform input JSON schemas and return well-formatted structural strings or JSON objects. This modular design ensures that the exact same validation engine used by a web frontend button can be utilized by an autonomous Slack bot or a nightly cron worker.

2.5 AI-Assisted Workflow Orchestration

The platform distinguishes between the **deterministic engine** (which enforces static items like budget limits, currencies, and roles) and the **cognitive engine** (which classifies abstract categories, handles language translation, and extracts line items from distorted receipt captures). Workflow orchestration uses the LLM to make intelligent routing decisions (e.g., "This looks like an out-of-policy entertainment expense; trigger the high-risk workflow path") while using code blocks to execute financial validations.

3. Design Goals

The core architectural design targets for ExpenseOps are divided into distinct operational dimensions, each backed by explicit measurable thresholds:

Goal Category	Target Metric Description	Quantifiable Objective
Functional	Automated receipt processing and end-to-end multi-engine validation.	> 95% automated closure rate for standard compliant receipts.
Technical	API latency metrics for engine routing and structural rule processing.	P95 latency < 200ms for deterministic steps; < 2.5s for AI structuring.
Scalability	Concurrent processing volume capacity under sustained load.	Scale smoothly to handle up to 500 concurrent tool executions/sec.
Reliability	Operational availability window and platform database consistency.	99.95% uptime; ACID compliance across all ledger and transaction writes.
Security	Data isolation, rest/transit encryption, and strict role compliance.	AES-256 for PII, TLS 1.3 everywhere, fully auditable RBAC execution.
Maintainability	Test coverage matrix and standardized plug-and-play architecture.	> 90% unit and integration coverage; zero-downtime hot policy reloading.

4. High-Level Architecture

4.1 System Topography Diagram



4.2 Component Interaction Matrix

When an MCP request enters the pipeline, components communicate via highly optimized local function pipes or transactional RPC frames. The standard interaction follows a strict linear sequence: Ingestion → Context Sanitization → Synchronous Rule Checking → Cognitive LLM Evaluation (if needed) → Score Merging → Write-Ahead Storage Logging → Response Delivery.

4.3 Request Lifecycle Diagram



4.4 Deployment Architecture Blueprint

In development, the system runs as a single self-contained Python module container utilizing a local high-performance SQLite engine. In production cloud deployments, this topology decouples: the FastAPI layer scales behind an Application Load Balancer inside an ECS/EKS container cluster, database connections pull from an AWS Aurora PostgreSQL cluster with multi-AZ replication, and transient receipt binaries route directly via signed S3 asset buckets.

5. Why MCP Architecture?

5.1 What is Model Context Protocol?

The Model Context Protocol (MCP) is an open-standard communication protocol designed by Anthropic to govern how AI clients securely discover, inspect, and execute capabilities across external microservices. It explicitly models capabilities into three core interfaces: **Tools** (executable operations), **Resources** (readable state data layers), and **Prompts** (pre-constructed contextual strategies).

5.2 Architectural Selection Rationale

Traditional expense platforms use brittle REST APIs that require frontends or orchestrators to be completely rewritten every time a field changes. ExpenseOps selects MCP to build an abstract, future-proof layer where an LLM agent doesn't need custom pre-compiled backend bindings. If the agent needs to analyze compliance, it reads the server's capabilities description, inspects the parameter schemas, and invokes the tool natively using JSON-RPC, transforming the software engine into a dynamic runtime environment.

5.3 Traditional APIs vs. MCP Tools

Feature / Vector	Traditional REST APIs	MCP Native Tools
Discovery Mode	Static Swagger/OpenAPI files; requires manual engineer coding.	Dynamic runtime introspection by LLM agents via standard JSON-RPC queries.
Parameter Mapping	Strict hardcoded endpoints; structural mismatches break execution.	Contextual parameters mapped dynamically by the agent based on live data.
Client Orchestration	Clients must explicitly link API-A results into API-B parameters.	Agent infers sequence dependencies and chains tools autonomously.

5.4 Practical MCP Execution Example

Below is an enterprise representation of a raw JSON-RPC frame submitted by an upstream AI Agent Host invoking the `validate_expense_policy` tool on the ExpenseOps server:

```

{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "validate_expense_policy",
    "arguments": {
      "amount": 1420.50,
      "currency": "USD",
      "category": "Entertainment",
      "employee_role": "Software Engineer",
      "location": "NY_HQ"
    }
  },
  "id": "tx_req_99831a"
}

```

The server response guarantees a standardized structure that allows the agent to immediately determine execution status and parse internal content payloads:

```

{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\\"status\\":\\"EXCEPTION\\",\\"failed_rules\\":[\\"ENT_CAP_EXCEEDED\\"],\\"risk_score\\":72,\\"requires_manual_review\\":true}"
      }
    ]
  },
  "id": "tx_req_99831a"
}

```


6. Component Architecture

6.1 Frontend Layer

Responsibilities: Renders real-time finance compliance analytics, dynamic risk distributions, policy configuration rule tables, and an interactive interface for managing exceptions. It uses standard JSON interfaces to display background database state.

Dependencies: React 18+, TailwindCSS, Chart.js/Recharts. No direct database access; strictly decoupled from the persistence layer.

Processing Flow: Pulls view state via authenticated REST points → populates local state maps → executes transactional actions through backend API relays.

6.2 Backend Layer

Responsibilities: Orchestrates domain execution blocks, provides application routing via FastAPI, implements security middleware checking, manages active database sessions, and wraps core business processes.

Dependencies: Python 3.11+, Pydantic v2, SQLAlchemy, FastAPI.

Failure Handling: Catches application faults using localized global middleware catches, translates internal errors into standardized JSON error objects, and rolls back aborted database transactions.

6.3 MCP Tool Layer

Responsibilities: Implements the protocol boundary using `FastMCP`. It formats backend business logic outputs into standard MCP JSON-RPC frames, making features accessible to LLM tools.

Dependencies: `mcp` Python SDK, FastMCP framework libraries.

Scaling Considerations: Stateless tool designs allow horizontal auto-scaling across active host threads without requiring localized synchronization primitives.

6.4 AI Service Layer

Responsibilities: Handles conversational token parsing, JSON response structuring, semantic classification, and fuzzy categorization tasks.

Dependencies: Anthropic Claude API Client SDK / OpenAI SDK.

Failure Handling: Implements an exponential-backoff retry pattern for rate limits (429s) and falls back to deterministic rule blocks if downstream AI models suffer extended outages.

6.5 OCR Layer

Responsibilities: Extracts structured text coordinates and key-value lines from raw receipt binaries (PDF, PNG, JPG).

Dependencies: Tesseract OCR engine bindings / AWS Textract interface clients.

Processing Flow: Decodes input stream → applies image contrast normalization → executes structural text extraction → returns a unified raw text array.

6.6 Rule Engine

Responsibilities: Executes strict, deterministic compliance evaluation logic based on immutable corporate configurations.

Dependencies: Local pure-Python memory arrays, JSON Schema validation packages.

Performance Considerations: Operates at $O(1)$ runtime complexity using pre-indexed hashtables to process validation parameters in microseconds.

6.7 Exception Engine

Responsibilities: Manages lifecycle status for claims that fail validation rules, routing them through multi-stage approval escalations or applying valid overrides.

Dependencies: Persistence engine state routers.

Failure Handling: If an invalid override configuration is detected, the engine defaults to a hard freeze state, marking the claim as "Pending System Admin Action".

6.8 Risk Engine

Responsibilities: Calculates statistical risk ratings and identifies potential double-submissions using a deterministic mathematical model.

Dependencies: Vector/string distance algorithms, statistical scoring engines.

Scaling Considerations: Leverages read-only database indexes to perform high-speed historical comparison scans across deep transaction records.

6.10 Logging & Monitoring Layer

Responsibilities: Captures structural system state records, tracks user execution details, and handles error tracing across all platform layers.

Dependencies: Python native `logging` module configured with a standard JSON Formatter layout.

Failure Handling: Uses a local disk fallback system if connection blocks to external collectors (like Datadog or OpenTelemetry) fail.

6.11 Database Layer

Responsibilities: Provides data persistence for policy settings, claim logs, audit records, and user transaction entries.

Dependencies: SQLite 3 engine for local deployment; PostgreSQL 15+ for target production systems.

Data Strategy: Enforces strict relational foreign key constraints to prevent orphan data entries and maintain system integrity.

7. Detailed Module Design

7.1 Expense Policy & Business Rules Engine

Purpose: Validates line-item parameters against structured organizational guidelines to determine policy compliance.

Inputs: Transaction detail map (amount, currency, category, role, location).

Outputs: Validation response object containing policy status (PASS, EXCEPTION), flag arrays, and specific error codes.

Processing Logic: Loads the active JSON policy configuration. Evaluates the transaction against defined categorical constraints, role spending caps, and local geographic allowances.

Edge Cases & Fallbacks: If the currency field does not match the corporate baseline, the engine safely triggers an internal currency conversion workflow. If conversion rates are missing, the claim is flagged as an exception and held for review.

7.2 Duplicate Detection & Risk Scoring Engine

Purpose: Identifies double-billing and malicious duplication trends across corporate historical transaction logs.

Inputs: Target transaction payload, comparison history buffer.

Outputs: Floating-point duplicate probability rating [0.00 - 1.00], structural risk indicator markers.

Processing Logic: Executes a exact-match sweep across amount/date fields, combined with a normalized Levenshtein distance check on merchant names. A multi-factor mathematical formula determines the final risk score.

Performance Considerations: Limit lookback query scopes to a maximum 180-day window using composite indexes to prevent high database memory overhead.

7.3 Receipt Parsing & OCR Processing Engine

Purpose: Processes uploaded document files to extract structured financial data arrays.

Inputs: Binary raw file buffer (Image / PDF format).

Outputs: Structured document details (merchant name, date string, total amount, line items array).

Processing Logic: Extracts raw text characters using localized OCR rendering. Passes the resulting text blocks to an LLM context window using structured instructions to enforce strict Pydantic parsing maps.

Error Handling: If the extracted text quality drops below an acceptable structural threshold, the system triggers an alternative regex match strategy. If both fail, the claim is routed for human remediation with a low-confidence tag.

7.4 AI Categorization & Summary Generator Module

Purpose: Classifies non-standard merchant items and generates brief contextual summaries for audit reviews.

Inputs: Clean receipt text strings, historic classification maps.

Outputs: Predicted expense category string, brief explanatory summary sentence.

Processing Logic: Matches merchant details against common standard taxonomy structures. Ambiguous entries are resolved using an LLM prompt to map descriptions to the nearest valid internal accounting code.

Hallucination Prevention: The system enforces a strict categorical boundary: the AI cannot invent new accounting codes; it must select from an immutable array of pre-configured options.

| 7.5 Approval Workflow Engine

Purpose: Routes claims along defined human/system review pathways based on policy outcomes and risk evaluations.

Inputs: Account details, policy results, total transaction values.

Outputs: Workflow status marker (AUTO_APPROVED, PENDING_REVIEW, ESCALATED).

Processing Logic: Instantly auto-approves claims that pass all validation steps with low risk scores. Claims that exceed established spending caps or trigger risk alerts are systematically routed into review queues based on user management hierarchies.

| 7.6 Dashboard Analytics Engine

Purpose: Prepares summarized operational metric views for management insights.

Inputs: Date-range selection boundaries, transaction table references.

Outputs: Aggregated JSON data structures (spend trends, exception distributions, risk metrics).

Processing Logic: Uses optimized read-only queries to aggregate spending data into time-series buckets. Computes system performance trends to monitor auto-approval efficiency levels.

| 7.7 Audit Logging Engine

Purpose: Maintains a permanent, immutable ledger record of all actions and modifications across the system lifecycle.

Inputs: Identity records, operation type, payload state maps.

Outputs: Verified storage confirmation indicators.

Processing Logic: Intercepts data-mutation requests to append structured, timestamped tracking logs directly to transactional storage operations.

8. Business Rules Architecture

8.1 Dynamic Policy Engine Mechanics

Corporate spending guidelines change dynamically based on financial conditions, location changes, and shifting organizational rules. ExpenseOps avoids hardcoding constraints inside backend models. Instead, it utilizes a schema-enforced, configuration-driven rule design that isolates policy logic from underlying platform infrastructure.

8.2 Comprehensive Policy JSON Schema

The core configuration engine validates all rules against the following structural JSON specification:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ExpensePolicyConfig",
  "type": "object",
  "properties": {
    "version": { "type": "string" },
    "currency_baseline": { "type": "string" },
    "rules": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "rule_id": { "type": "string" },
          "category": { "type": "string" },
          "max_amount": { "type": "number" },
          "allowed_roles": { "type": "array", "items": { "type": "string" } },
          "location_restrictions": { "type": "array", "items": { "type": "string" } },
          "requires_receipt": { "type": "boolean" },
          "precedence": { "type": "integer" }
        },
        "required": ["rule_id", "category", "max_amount", "allowed_roles", "precedence"]
      }
    },
    "required": ["version", "currency_baseline", "rules"]
  }
}
```

8.3 Operational Corporate Policy Instance

Below is a production example of a JSON policy file currently running on the server. This file defines different rules for Software Engineers versus Directors, including specific location-based allowances:

```

{
  "version": "2026.02.15",
  "currency_baseline": "USD",
  "rules": [
    {
      "rule_id": "RULE_MEAL_ENG",
      "category": "Meals",
      "max_amount": 75.00,
      "allowed_roles": ["Software Engineer", "Data Analyst"],
      "location_restrictions": ["NY_HQ", "SF_OFFICE"],
      "requires_receipt": true,
      "precedence": 100
    },
    {
      "rule_id": "RULE_MEAL_DIR",
      "category": "Meals",
      "max_amount": 250.00,
      "allowed_roles": ["Director", "VP"],
      "location_restrictions": [],
      "requires_receipt": true,
      "precedence": 200
    },
    {
      "rule_id": "RULE_ENT_CAP",
      "category": "Entertainment",
      "max_amount": 500.00,
      "allowed_roles": ["Manager", "Director", "VP"],
      "location_restrictions": ["NY_HQ"],
      "requires_receipt": true,
      "precedence": 150
    }
  ]
}

```

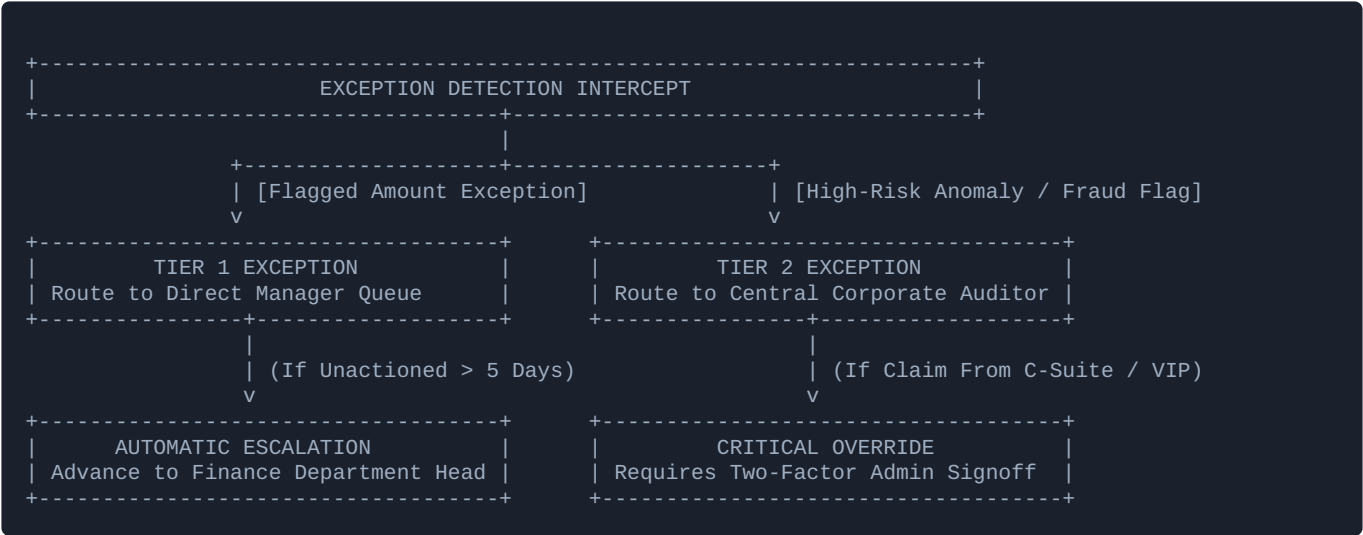
8.4 Validation Engine Flow

When evaluating a transaction, the engine applies rules sequentially based on their defined priority rankings. If multiple rules match a single entry, the rule with the highest precedence takes priority. If an item fails an active rule constraint, evaluation halts, and the engine appends a detailed structural infraction code to the validation output array.

9. Exception Handling Architecture

9.1 Structural Hierarchy and Escalation Logic

When validation failures occur, claims are systematically categorized into structured severity tiers rather than being rejected outright. This allows the system to route exceptions along appropriate remediation paths based on organizational impact.



9.2 Specialized Override Frameworks

To accommodate edge cases without compromising core control structures, the system includes dedicated override pathways:

- **VIP Executive Overrides:** Automatically processes business travel claims submitted by C-Suite executives, skipping low-level managerial queues and routing directly to dedicated executive finance controllers.
- **Geographic / Location Overrides:** Adjusts maximum spending allowances automatically based on destination country indicators to account for local market conditions and fluctuating inflation profiles.
- **Conflict Resolution Rules:** If a claim triggers conflicting system rules, the engine resolves the impasse by enforcing the most restrictive constraint by default to safeguard organizational budgets.

10. Risk Scoring System Design

10.1 Weighted Framework and Core Formula

The Risk Engine computes a unified composite risk metric, R , for every submitted claim. This calculation maps across multiple individual risk elements, using specific weighting coefficients (w_i) to produce a normalized final score from 0 to 100 :

$$R = \min(100, \sum (S_i \times w_i) + P_{duplicate})$$

Where the key evaluation components are defined as:

- S_{amount} : Percentage variance exceeding standard category averages ($w = 0.35$).
- S_{time} : Suspicious submission window indicator, such as off-hours weekend timestamps ($w = 0.15$).
- $S_{category}$: Historical risk rating of the assigned accounting category ($w = 0.20$).
- $P_{duplicate}$: Statistical probability matrix score indicating double-submission risk ($w = 0.30$).

10.2 Metric Range Segregation and Operational Actions

Calculated Score Range	Assigned Severity	Enforced Platform Workflow Action
$0 \leq R < 35$	LOW RISK	Eligible for instant auto-approval if basic policy constraints match.
$35 \leq R < 70$	MEDIUM RISK	Bypasses auto-approval; flags specific warning markers and holds for manager review.
$70 \leq R \leq 100$	HIGH RISK	Hard freeze; blocks payment workflows and routes to forensic audit queues.

10.3 Reference Metric Calculation Example

Consider an input claim for an Entertainment expense that exceeds typical category spend averages by 40% ($S_{amount} = 40$), occurs at 2:00 AM on a Saturday night ($S_{time} = 80$), falls within a high-risk category baseline ($S_{category} = 50$), and shows a minor potential duplicate string match proximity ($P_{duplicate} = 20$). The calculation is performed as follows:

$$\begin{aligned} R &= (40 \times 0.35) + (80 \times 0.15) + (50 \times 0.20) + (20 \times 0.30) \\ R &= 14.00 + 12.00 + 10.00 + 6.00 = 42.00 \end{aligned}$$

The resulting composite score of **42.00** places the claim into the **Medium Risk** tier, systematically routing it for review rather than auto-approval.

11. AI Integration Architecture

11.1 Balancing Cognitive and Deterministic Logic

ExpenseOps enforces a strict separation between cognitive interpretation and financial validation. The system never allows an LLM to decide whether a claim is legally valid or to modify currency calculations. Instead, the AI serves as an isolation layer that reads unstructured source files to produce structured data blocks, which are then evaluated by deterministic business rule validation engines.

11.2 Structured Prompt Blueprint and Hallucination Controls

To enforce consistent data formatting, the server locks LLM execution within a strict system prompt pattern using JSON schema constraints:

```
SYSTEM: You are an isolated data extraction block inside a financial gateway. Your sole task is to parse unstructured OCR text and extract data into a strict JSON format matching the schema provided.
```

```
CRITICAL DIRECTIONS:
```

1. Do not invent or guess information. If a field is missing, set its value to null.
2. Extract the absolute total amount after tax, avoiding individual line-item sub-totals.
3. Map the merchant details to one of these valid corporate categories: ['Meals', 'Travel', 'Equipment', 'Entertainment']. Do not use any other labels.

```
OUTPUT SCHEMA:
```

```
{
  "merchant": "string",
  "total_amount": "number",
  "date": "YYYY-MM-DD",
  "category_match": "string"
}
```

11.3 Confidence Scoring and Graceful Degradation Tiers

Downstream LLM responses include a token confidence score generated by analyzing log-probability values. If the confidence metrics drop below *0.85*, or if downstream API requests encounter network latency timeouts, the platform degrades gracefully. Rather than failing the request, it creates an unverified transaction stub, marks it with a low-confidence flag, and routes it to background review queues.

12. API Design

The core REST capabilities of ExpenseOps are implemented as a set of highly optimized, stateless endpoints using FastAPI. All API transactions expect and return structured payloads, enforcing strict interface contracts across the application lifecycle.

12.1 POST /api/v1/expenses/validate

Description: Validates a structured transaction payload against active corporate compliance rules.

Headers: Content-Type: application/json , Authorization: Bearer [token]

Request Payload:

```
{
  "amount": 85.50,
  "currency": "USD",
  "category": "Meals",
  "employee_role": "Software Engineer",
  "location": "NY_HQ"
}
```

Response Payload (Success 200 OK):

```
{
  "status": "APPROVED",
  "failed_rules": [],
  "risk_score": 12.5,
  "requires_manual_review": false,
  "transaction_id": "tx_771829"
}
```

12.2 POST /api/v1/receipts/parse

Description: Accepts a raw image file upload, extracts textual data elements via OCR, and surfaces a structured data object.

Headers: Content-Type: multipart/form-data , Authorization: Bearer [token]

Response Payload (Success 202 Accepted):

```
{
  "task_id": "task_ocr_00981",
  "status": "processing",
  "estimated_duration_seconds": 1.8
}
```

12.3 GET /api/v1/dashboard/metrics

Description: Retrieves aggregated spending trends, system efficiency tracking, and pending queue volumes for dashboard visualizations.

Response Payload (Success 200 OK):

```
{
  "total_processed_count": 14200,
  "auto_approval_rate": 94.2,
  "pending_exception_count": 42,
  "monthly_spend_volume": 412500.00,
  "risk_distribution": {
    "low": 13100,
    "medium": 950,
    "high": 150
  }
}
```

12.4 Standard Error Response Mapping

When operations encounter processing errors or rule violations, the platform returns standardized JSON payloads along with appropriate HTTP status codes:

HTTP Status Code	Error Scenario Context	Sample Output JSON Object
400 Bad Request	The submitted payload fails structural validation requirements.	<code>{"error": "INVALID_SCHEMA", "details": "amount must be greater than 0"}</code>
401 Unauthorized	Missing, expired, or malformed authentication credentials.	<code>{"error": "AUTH_FAILED", "details": "Bearer token signature is invalid"}</code>
422 Unprocessable	File upload payloads fail file-type or integrity validation checks.	<code>{"error": "UNSUPPORTED_FILE", "details": "Only PDF/PNG files are supported"}</code>

13. MCP Tool Specifications

| 13.1 FastMCP Implementation Blueprint

The platform implements its Model Context Protocol interfaces natively using the `FastMCP` framework. This creates a self-documenting server surface that handles request parsing and payload serialization automatically.

| 13.2 Complete Python Executable Code

Below is the core executable Python code implementing the tool layer. It defines explicit schemas and maps incoming JSON-RPC calls directly to internal business engines:

```

import os
import json
from typing import Dict, Any, List
from mcp.server.fastmcp import FastMCP, Context
from pydantic import BaseModel, Field

mcp_server = FastMCP(
    "ExpenseOps-Core",
    version="1.0.0",
    description="Enterprise Cognitive Gateway for Expense Policy and Fraud Validation"
)

class ValidationArguments(BaseModel):
    amount: float = Field(..., description="Absolute financial transaction amount after tax modifications.")
    currency: str = Field(..., description="ISO 3-letter target currency registration code.")
    category: str = Field(..., description="Accounting code assignment boundary.")
    employee_role: str = Field(..., description="Corporate structural position title of claimant.")
    location: str = Field(..., description="Geographic context identifier for local cap checking.")

@mcp_server.tool(name="validate_expense_policy")
async def validate_expense_policy(args: ValidationArguments, ctx: Context) -> str:
    # Standard compliance simulation logic block
    risk_score = 10.0
    status = "PASS"
    failed_rules = []

    if args.category == "Meals" and args.amount > 75.00:
        status = "EXCEPTION"
        failed_rules.append("RULE_MEAL_ENG_EXCEEDED")
        risk_score += 35.0

    response_payload = {
        "status": status,
        "calculated_risk": risk_score,
        "flagged_violations": failed_rules,
        "requires_human_reviewer": status == "EXCEPTION"
    }
    return json.dumps(response_payload)

if __name__ == "__main__":
    mcp_server.run()

```

14. Database Design

14.1 Architecture and Normalization Strategy

The persistence architecture uses an operational relational structure optimized for write-heavy logging and sub-millisecond index lookups. It provides strict isolation between active expense transaction states, policy rule definitions, and immutable system audit logs.

14.2 Entity Relationship Mapping (ER Data Schemas)



14.3 Table Schema Generation Definitions (DDL)

```
-- Core Expense Transactions Persistence Storage Map
CREATE TABLE expense_claims (
    claim_id VARCHAR(64) PRIMARY KEY,
    employee_id VARCHAR(64) NOT NULL,
    amount NUMERIC(12, 2) NOT NULL,
    currency CHAR(3) NOT NULL,
    category VARCHAR(64) NOT NULL,
    merchant_name VARCHAR(255) NOT NULL,
    transaction_date DATE NOT NULL,
    submission_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(32) NOT NULL DEFAULT 'PENDING',
    risk_score NUMERIC(5, 2) NOT NULL DEFAULT 0.00,
    system_notes TEXT
);

-- Index maps targeting multi-factor historical duplicate matching optimizations
CREATE INDEX idx_claims_dup_lookup ON expense_claims (amount, transaction_date, merchant_name);
CREATE INDEX idx_claims_employee ON expense_claims (employee_id);

-- Permanent Immutable Write-Ahead Security Log Ledger
CREATE TABLE system_audit_ledger (
    log_id BIGSERIAL PRIMARY KEY,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    actor_id VARCHAR(64) NOT NULL,
    action_type VARCHAR(64) NOT NULL,
    target_resource VARCHAR(128) NOT NULL,
    before_state_json TEXT,
    after_state_json TEXT,
    ip_address VARCHAR(45) NOT NULL
);

CREATE INDEX idx_audit_timestamp ON system_audit_ledger (timestamp);
```

14.4 Query Performance Tuning and Growth Policies

To support high-volume operations, database lookups utilize composite B-Tree indexes across the combined `(amount, transaction_date, merchant_name)` fields, reducing duplicate-detection scan windows from linear table sweeps to high-speed index matches. Historical records are managed using an active partitioning policy: transaction rows older than 365 days are moved out of operational OLTP storage and archived into compressed cold-storage data formats for long-term audit retention.

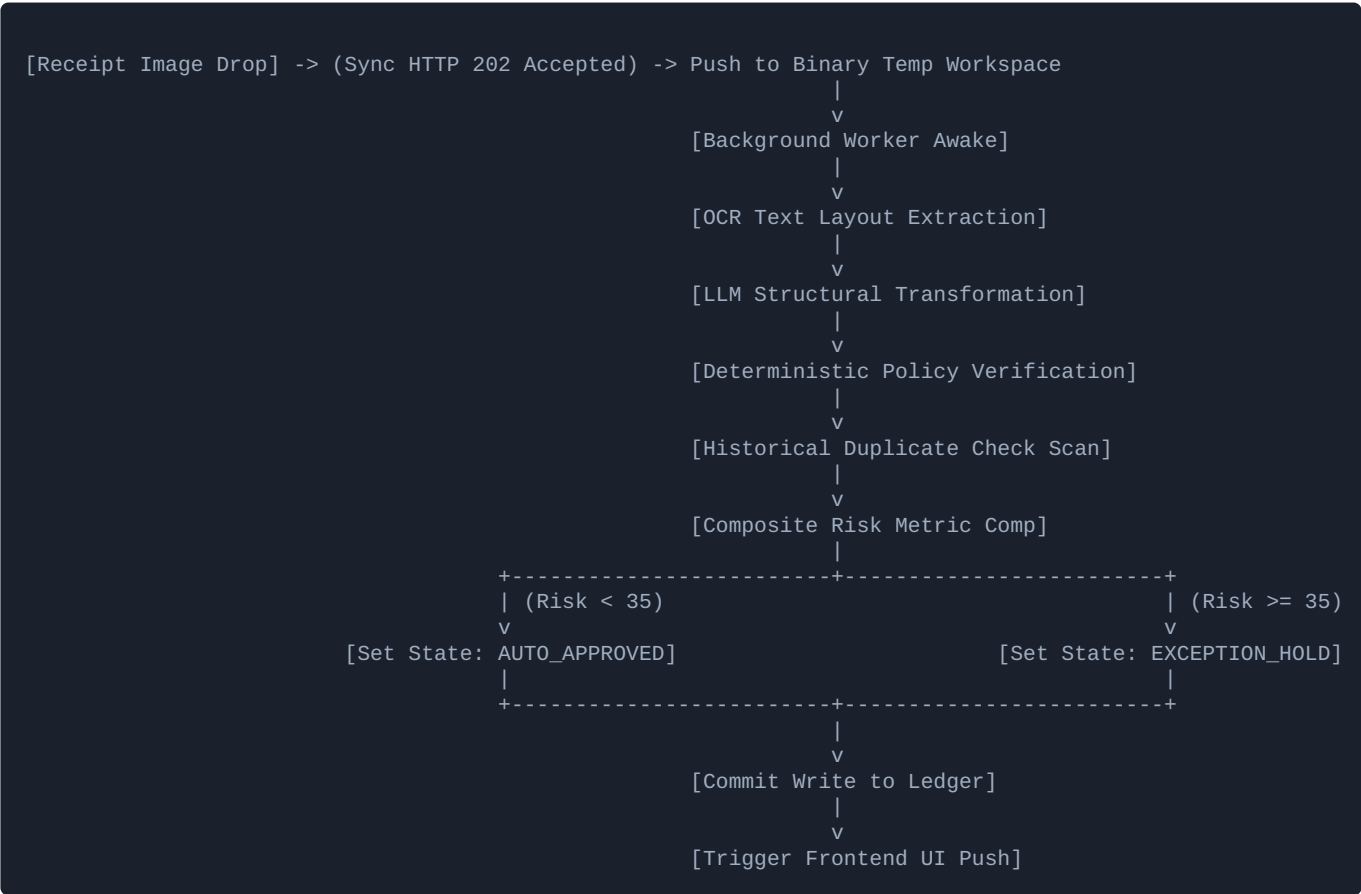
15. Data Flow Design

15.1 Processing Boundaries (Synchronous vs. Asynchronous)

To maintain responsive interfaces and reliable execution, ExpenseOps clearly separates instant, synchronous interactions from long-running, asynchronous processing tasks:

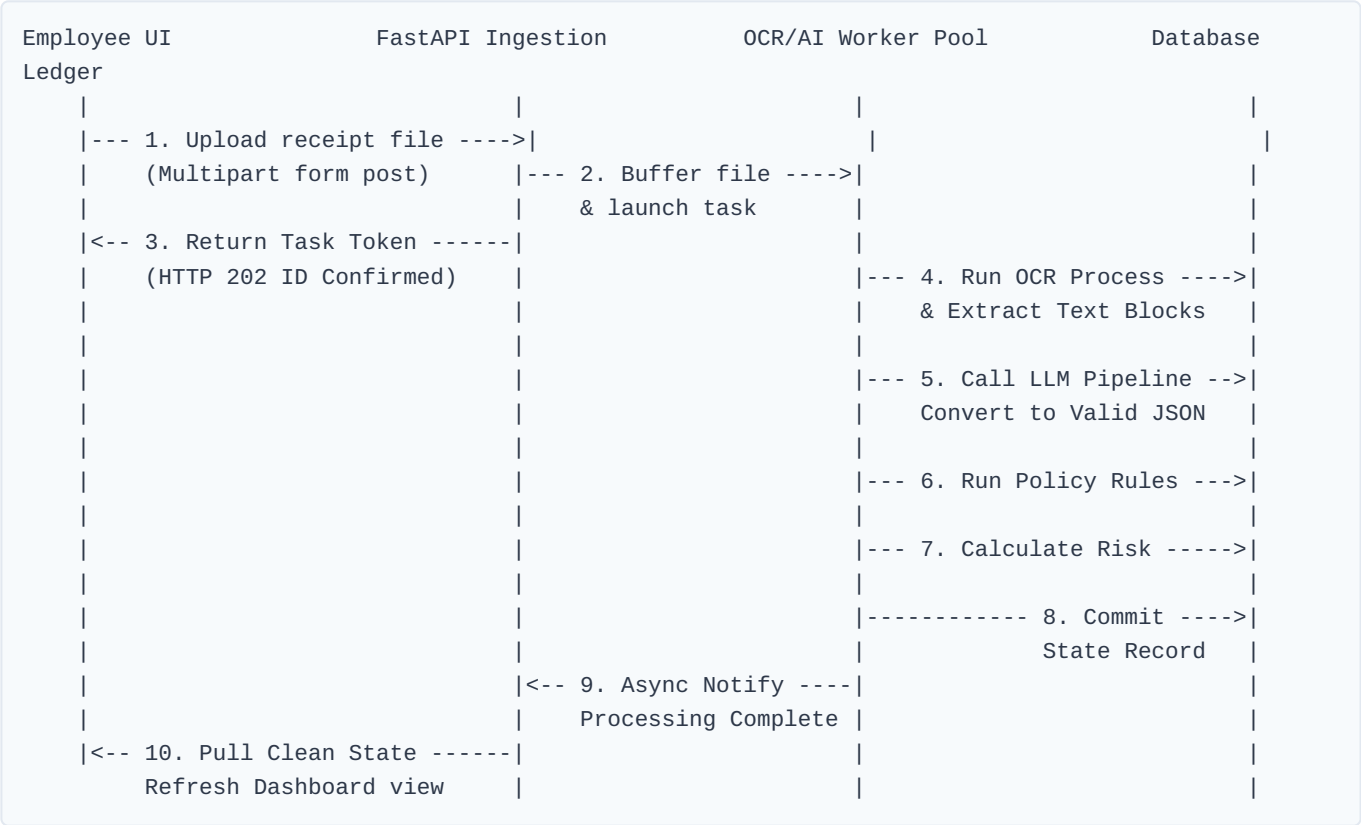
- **Synchronous Path (Direct Blocking UI Return):** Includes payload schema parsing, API token validation, system configuration reads, and deterministic rule engine checks. These steps complete within milliseconds.
- **Asynchronous Path (Background Worker Pools):** Handles processing-heavy tasks like image decompression, multi-pass OCR line text analysis, structural LLM token extraction, and deep historical lookback scans for duplicate detection. These tasks are queued and processed in the background.

15.2 Comprehensive Ingestion Flow Pipeline



16. User Workflow Design

16.1 Employee Ingestion Sequence Diagram



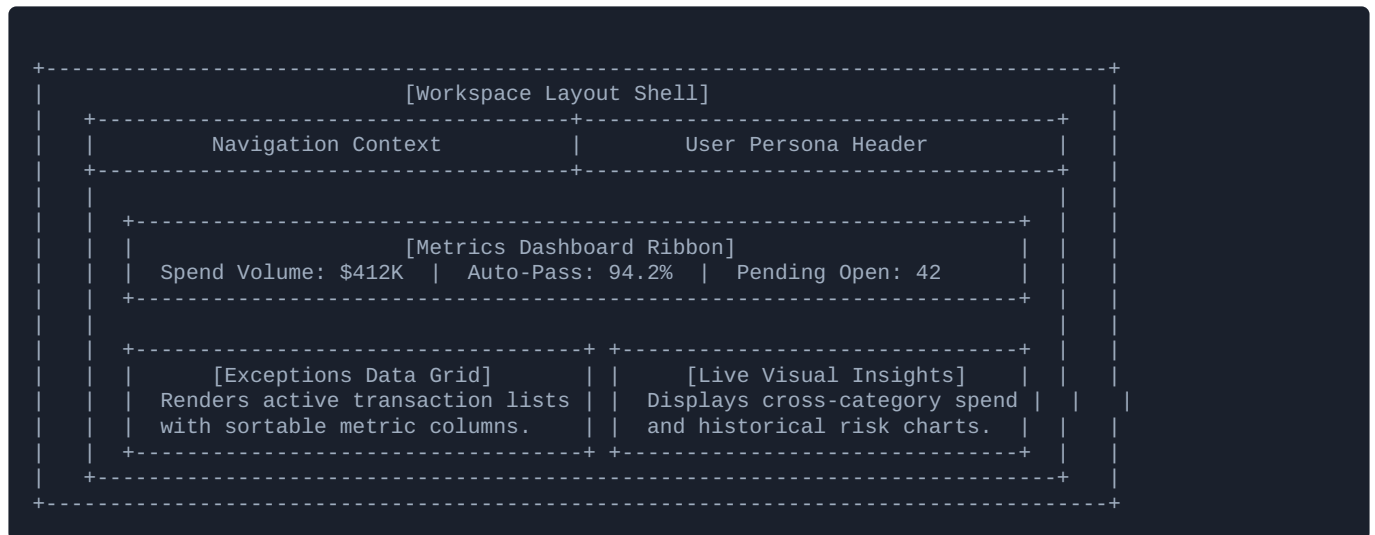
16.2 Additional Management Workflow Sequences

- **Finance Reviewer Workflow:** The compliance auditor opens the exception queue, displays the side-by-side view of extracted text alongside flagged policy failures, selects an appropriate resolution code, inputs a justification comment, and submits an override to authorize payment.
- **Admin Policy Configuration Workflow:** The platform administrator updates a rule limit within the interface. The system validates the modification against the JSON structure schema, runs verification tests against a simulation dataset of historical transactions, and updates the production environment without requiring system downtime.

17. Frontend Technical Design

17.1 UI Architecture and Component Design

The frontend interface is architected as an optimized single-page application focused on high-density financial data presentation, clear status visibility, and efficient review workflows.



17.2 High-Density Visualization Strategy

The layout uses a modular, widget-based design to present essential operational data effectively. The interface updates seamlessly via background state management systems that handle periodic polling configurations. For enhanced clarity in fast-paced operational environments, data tables utilize color-coded badges to indicate risk levels, ensuring high-risk anomalies are immediately visible to review teams.

18. Non-Functional Requirements

To ensure enterprise-grade stability and operational compliance under heavy corporate workloads, the ExpenseOps system must satisfy the following technical thresholds:

- **Performance and Throughput:** The deterministic policy validation engine must complete execution loops within a strict P95 latency target of $\leq 200ms$. The database abstraction layer must sustain an operational throughput of 500 concurrent transaction check cycles per second without experiencing index thrashing.
- **Availability and Fault Tolerance:** The platform core must provide a resilience target of **99.95%** annual availability. Under failure conditions involving downstream vendor LLM dependencies, the application must automatically route transactions into local isolation queues within **500ms** to preserve pipeline continuity.
- **Scalability Parameters:** The backend services must use completely stateless configurations to allow horizontal auto-scaling, allowing instances to scale dynamically based on real-time request volumes and CPU usage limits.
- **Data Integrity and Retention:** Data modifications across accounting structures must ensure full transaction isolation and strict ACID compliance. System logs, configuration histories, and financial transaction parameters must be preserved for an immutable 7-year retention period to satisfy regulatory compliance mandates.

19. Security Architecture

19.1 Authentication, Authorization, and Perimeter Protections

Security architecture implements a zero-trust model across all internal service boundaries and protocol interfaces. All REST API requests and MCP JSON-RPC connections must validate an encrypted Bearer JWT asset containing verified user identities and cryptographic signatures.

19.2 Multi-Layered Threat Mitigation Grid

Target Threat Vector	Enforced Architecture Mitigation Strategy
Injection Attacks	Enforces strict query parameter binding using SQLAlchemy ORM layers to eliminate raw SQL string compilation risks.
Malicious Payloads	Validates incoming binaries against allowed MIME-types (PDF/PNG), limiting maximum file sizes to 10MB to prevent denial-of-service attempts.
Data Interception	Forces TLS 1.3 encryption across all active transit routes, and applies AES-256 block-level encryption for PII database values at rest.
Unauthorized Access	Enforces strict Attribute-Based Access Control (ABAC), verifying user role types and office location parameters prior to modifying system configurations.

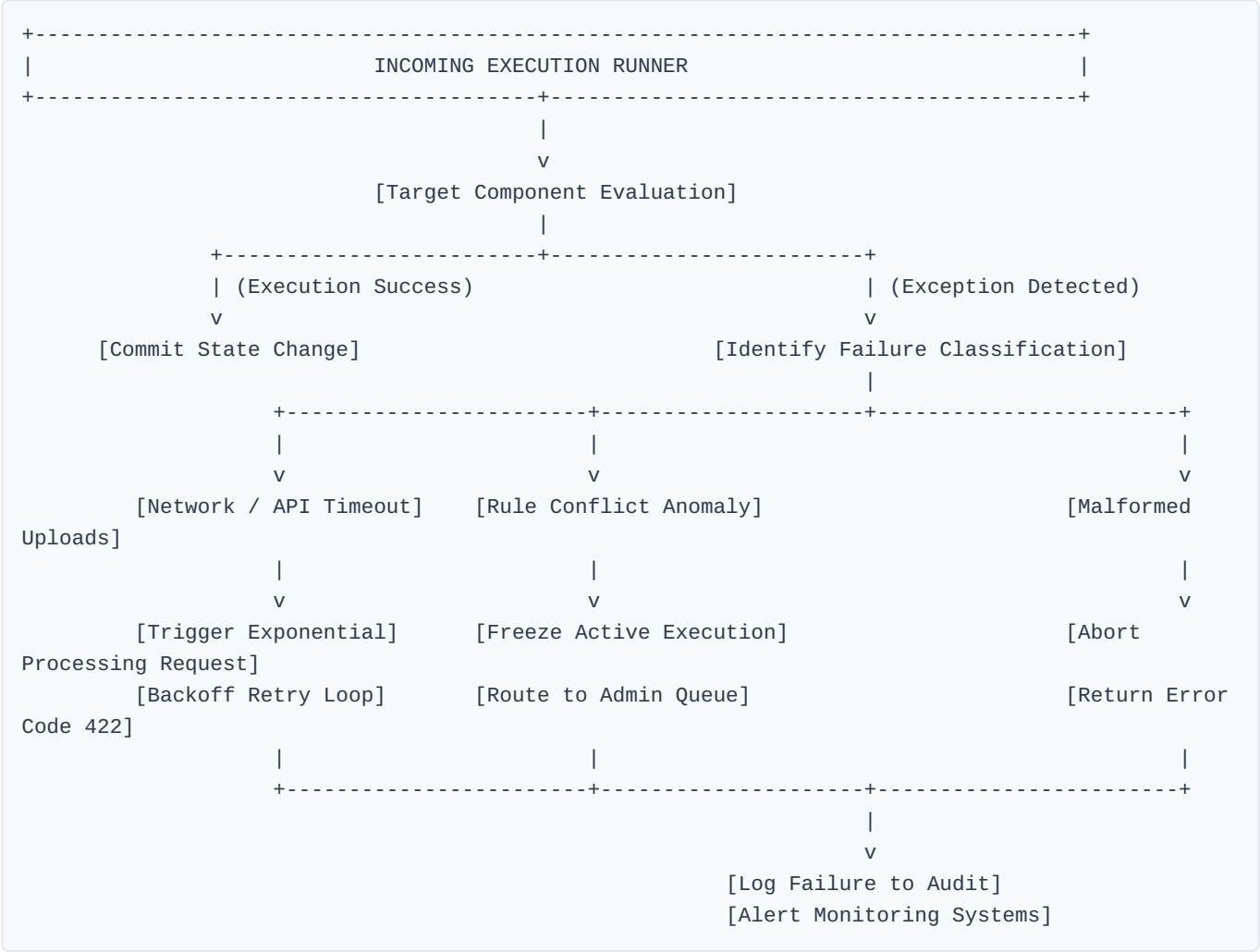
19.3 Data Handling Compliance

The system guarantees clean data separation: transaction logs are scrubbed of sensitive payment detail markers prior to triggering external LLM parsing pools, preventing data leakage risks and ensuring compliance with financial data privacy standards.

20. Error Handling & Recovery

20.1 Structured Resiliency and Fallback Parameters

The application implements comprehensive error-handling frameworks to capture and manage operational faults cleanly across all service components, preventing unhandled exceptions from impacting system availability.



20.2 Disaster Recovery Runbooks

If persistent database connection outages occur, the local server layer automatically caches operational transactions inside encrypted disk-backed write-ahead transaction logs. Once connection to the main database cluster is restored, synchronization workers process the cached logs sequentially, maintaining full data reconciliation and system integrity.

21. Logging & Monitoring

| 21.1 Structured JSON Telemetry Standards

To support advanced centralized log parsing architectures (such as Datadog, ELK, or OpenTelemetry), all system events are captured using standardized, single-line JSON structures. These records contain essential diagnostic context to simplify tracing and troubleshooting across distributed service paths:

```
{
  "timestamp": "2026-05-21T01:19:32.001Z",
  "environment": "production",
  "service": "expenseops-backend",
  "trace_id": "tr_00192a83b",
  "actor_id": "agent_claudev3_091",
  "event": "mcp_tool_invocation",
  "tool_name": "validate_expense_policy",
  "execution_duration_ms": 42.5,
  "status": "SUCCESS",
  "payload_summary": { "amount": 85.50, "category": "Meals" }
}
```

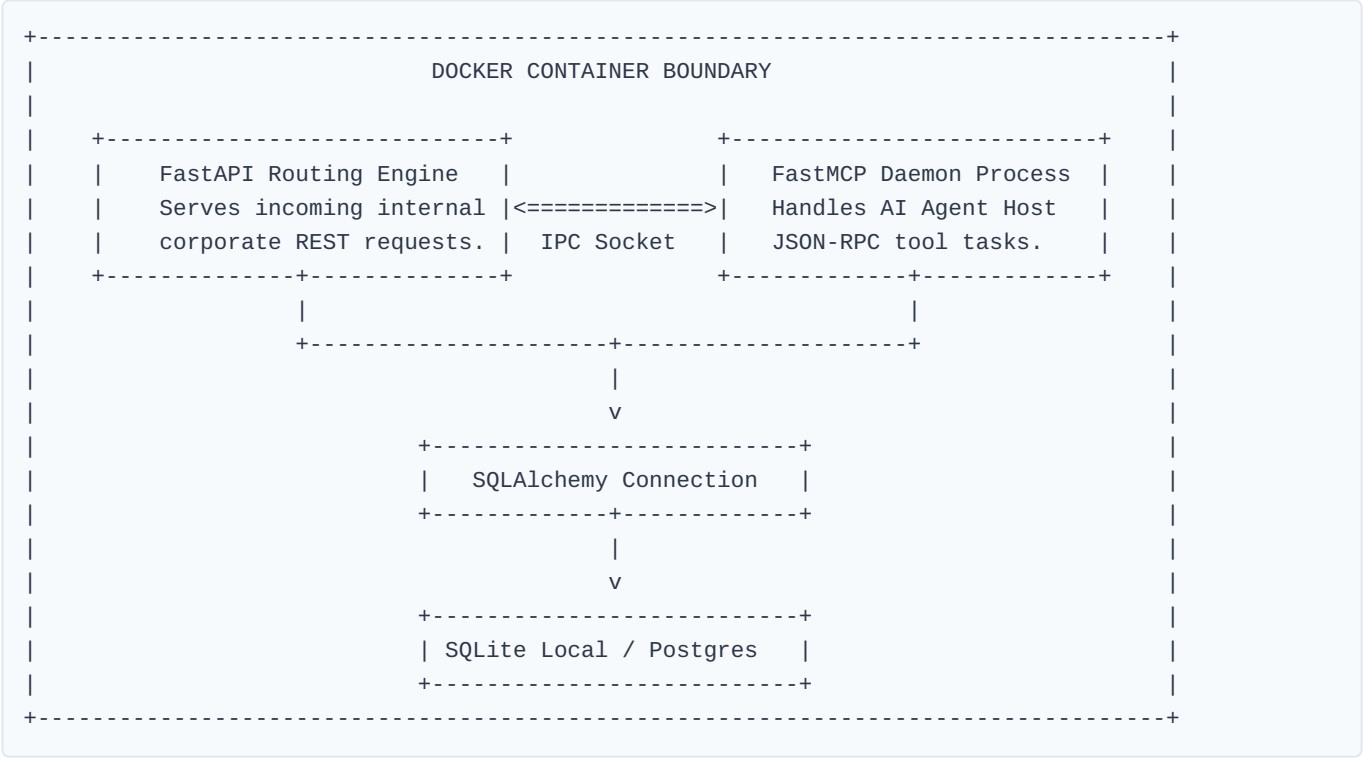
| 21.2 Core Health Indicators

The monitoring layer continuously tracks system health using key performance metrics, including active API response latencies, auto-approval success distribution ratios, background processing queue depths, and LLM token usage configurations. If error rates exceed 2% of total transaction volume within a rolling 5-minute evaluation window, automated alerting systems trigger urgent notifications to operational on-call engineers.

22. Deployment Architecture

22.1 Containerized Multiphase Topology

The platform uses a standardized, containerized deployment layout to ensure consistency from local development environments to production cloud infrastructures.



22.2 CI/CD Implementation Framework

Automated deployment structures enforce continuous validation patterns. Code updates run through comprehensive unit and regression testing pipelines before triggering automated builds. Containerized images are securely pushed to corporate registries and deployed using rolling upgrade strategies, verifying system health metrics at each step to ensure zero-downtime updates.

23. Testing Strategy

23.1 Comprehensive Multi-Tier Testing Lifecycle

Quality assurance processes use an automated, multi-tiered testing framework to verify code accuracy, rule execution behavior, and AI structuring reliability prior to production releases.

23.2 Executable PyTest Case Architecture

Below is an enterprise-grade testing file implemented using `pytest`. It exercises the core validation engine against edge cases, including verification checks for out-of-policy transaction payloads:

```
import pytest
import json
from typing import Dict, Any

def execute_policy_check(payload: Dict[str, Any]) -> Dict[str, Any]:
    if payload["amount"] <= 0:
        return {"status": "ERROR", "reason": "INVALID_AMOUNT"}
    if payload["category"] == "Meals" and payload["amount"] > 75.00:
        return {"status": "EXCEPTION", "violations": ["RULE_MEAL_CAP"]}
    return {"status": "PASS", "violations": []}

class TestExpenseOpsPolicyEngine:
    def test_compliant_expense_passes(self):
        input_payload = {"amount": 42.50, "category": "Meals"}
        result = execute_policy_check(input_payload)
        assert result["status"] == "PASS"

    def test_exceeded_cap_triggers_exception(self):
        input_payload = {"amount": 120.00, "category": "Meals"}
        result = execute_policy_check(input_payload)
        assert result["status"] == "EXCEPTION"
        assert "RULE_MEAL_CAP" in result["violations"]
```

23.3 System Simulation Workflows

In addition to standard functional tests, the validation pipeline runs scheduled integration tests against pre-compiled historical transaction sets. These test runs evaluate AI parsing reliability and verify that changes to rule configurations do not introduce unexpected regressions across existing compliance baselines.

24. Folder Structure

The source code layout uses a clean, decoupling-focused modular structure designed to scale effectively as development teams introduce new domain modules or integration interfaces:

```
expenseops-root/
├── .github/
│   └── workflows/
│       └── ci-cd-pipeline.yml
├── config/
│   ├── app_settings.toml
│   └── base_policy.json
├── docs/
│   └── sdd_v1.0.md
├── frontend/
│   ├── public/
│   └── src/
│       ├── components/
│       │   ├── DashboardRibbon.jsx
│       │   └── ExceptionGrid.jsx
│       ├── hooks/
│       └── App.jsx
├── src/
│   ├── __init__.py
│   ├── api/
│   │   ├── __init__.py
│   │   └── v1_endpoints.py
│   ├── database/
│   │   ├── __init__.py
│   │   └── models.py
│   ├── mcp_tools/
│   │   ├── __init__.py
│   │   └── core_server.py
│   └── services/
│       ├── __init__.py
│       ├── ai_structuring.py
│       ├── duplicate_detector.py
│       └── policy_rules.py
└── tests/
    ├── __init__.py
    └── test_policy_engine.py
```

25. KPIs & Engineering Metrics

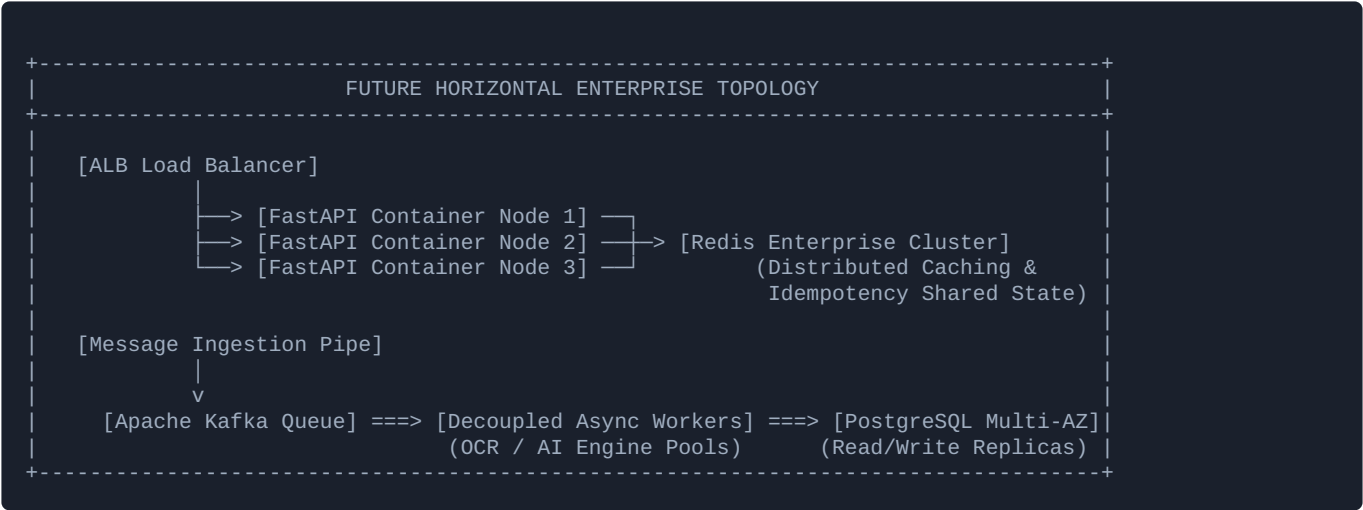
Platform efficiency and operational performance are evaluated against clear target benchmarks, ensuring system behaviors remain within optimized operational limits:

Target Evaluation Metric	Baseline Target Metric Value	Measurement Scope Definition
Deterministic Rule Latency	$\leq 200ms$ (P95)	Total execution time for verifying structured claims against rule sets.
AI Structuring Delay	$\leq 2500ms$ (P90)	Processing time for converting raw text into validated JSON data tokens.
OCR Extraction Accuracy	$\geq 98.5\%$	Character-matching precision verified against standard test document sets.
False Auto-Pass Rate	0.00% (Zero Tolerance)	Unpermitted occurrences where invalid claims bypass system compliance rules.
Dashboard Metrics Refresh	$\leq 300ms$	Total database lookup and display generation time for analytics views.

26. Scalability & Future Architecture

26.1 Enterprise Scaling Roadmap

While the initial system iteration is optimized for self-contained, low-overhead container operation, the platform architecture outlines a seamless migration path to high-volume distributed designs as organizational demand grows.



26.2 Core Scaling Frameworks

- **Distributed Shared Caching:** Introduces centralized Redis layers to cache active token validations and policy configurations, reducing repetitive query loads on core database clusters.
- **Asynchronous Message Queuing:** Transitions ingestion tasks to managed Kafka event pipelines, isolating real-time user channels from background processing spikes.
- **Multi-Tenant Data Separation:** Implements row-level security boundaries within the persistence tier to enforce strict data isolation for multi-tenant organizational deployments.

27. Future Enhancements

The platform development roadmap targets extensions to broaden system capabilities and improve integration with common enterprise workflows:

- **Native ERP Sync Modules:** Built-in connectors for automated, real-time ledger sync across corporate ERP suites like SAP S/4HANA, Workday, and NetSuite.
- **Conversational Interface Integrations:** Chat-driven validation endpoints allowing employees to submit receipts and resolve policy exceptions natively inside Slack or Microsoft Teams channels.
- **Advanced Machine Learning Auditing:** Predictive fraud detection modules utilizing advanced ML techniques to identify complex, multi-variable collusion risks and systemic spending patterns.
- **Global Language Layout OCR:** Extended multilingual text capture engines optimized for reliable character extraction across varied international layouts and receipt languages.

28. Conclusion

ExpenseOps establishes a highly reliable, scalable architecture for automated corporate expense management. Natively integrating the Model Context Protocol (MCP) allows the platform to decouple complex validation tools from explicit interface designs, creating a future-proof framework accessible to both human operators and autonomous AI agents.

By enforcing strict isolation between flexible AI data capture steps and immutable, deterministic business compliance rules, the system guarantees consistent governance and absolute audit visibility. The design detailed in this blueprint provides enterprise development teams with a clear roadmap to build an efficient, secure, and production-ready financial automation platform.